Mini Project II Report

on

VaaniSetu AI

Submitted in partial fulfillment of
the requirements for the Third Year of

Bachelor of Technology

in

Computer Engineering

Submitted by

Kartiki Raghuwanshi

Pranjali Mahajan

Vishwajeet Phapal

Sujal Mohite

DEPARTMENT OF COMPUTER ENGINEERING
S.S.V.P.S.'s B.S. DEORE COLLEGE OF ENGINEERING, DHULE
2025-2026

Mini Project II Report

on

VaaniSetu AI

Submitted in partial fulfillment of
the requirements for the Third Year of

Bachelor of Technology

in

Computer Engineering

Submitted by

Kartiki Raghuwanshi

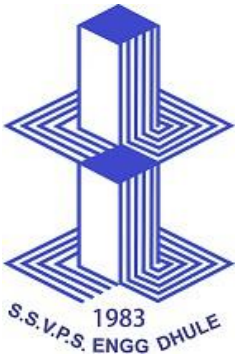
Pranjali Mahajan

Vishwajeet Phapal

Sujal Mohite

Guided By

Prof. Dr. R.V. Patil



DEPARTMENT OF COMPUTER ENGINEERING
S.S.V.P.S.'s B.S. DEORE COLLEGE OF ENGINEERING, DHULE

2025-2026

S.S.V.P.S.'s B.S. DEORE COLLEGE OF ENGINEERING, DHULE

DEPARTMENT OF COMPUTER ENGINEERING

Academic Year: 2025-2026

CERTIFICATE

This is to certify that the Mini Project II entitled **"VaaniSetu AI: A Multilingual Domain-Agnostic Conversational Assistant"** has been carried out by

Kartiki Raghuwanshi

Pranjali Mahajan

Vishwajeet Phapal

Sujal Mohite

under my guidance in partial fulfillment of the Third Year of Bachelor of Technology in Computer Engineering of Dr. Babasaheb Ambedkar Technological University, Lonere during the academic year 2025-2026. To the best of my knowledge and belief this work has not been submitted elsewhere for the award of any other degree.

Date: _____ Place: Dhule

Guide

Dr. R. V. Patil

Head of Department
Dr. I. S. Borse

Principal
Dr. Hitendra D. Patil

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who supported and guided us throughout the development of VaaniSetu AI.

We are deeply grateful to our project guide, Dr. R. V. Patil, for his invaluable guidance, constructive feedback, and constant encouragement at every stage of this work. His domain expertise in NLP and AI systems greatly shaped the direction of this project.

We extend our heartfelt thanks to Dr. I. S. Borse, Head of the Department of Computer Engineering, for providing the necessary resources and a research-oriented environment that made this work possible.

We are also thankful to the Principal, Dr. Hitendra D. Patil, for his encouragement and for fostering an innovative culture at S.S.V.P.S.'s B.S. Deore College of Engineering, Dhule.

Our sincere thanks go to the faculty members of the Department of Computer Engineering for their academic guidance and support throughout our undergraduate journey.

We acknowledge the contributions of the open-source NLP community the developers of Hugging Face Transformers, FastAPI, and the Bhashini translation ecosystem whose tools formed the backbone of this system.

Finally, we are profoundly grateful to our families and friends for their unwavering patience, motivation, and support throughout this project.

Kartiki Raghuwanshi

Pranjali Mahajan

Vishwajeet Phapal

Sujal Mohite

TABLE OF CONTENTS

FIGURE INDEX

TABLE INDEX

ABBREVIATIONS

Abbreviation	Full Form
NLP	Natural Language Processing
AI	Artificial Intelligence
ML	Machine Learning
API	Application Programming Interface
FAQ	Frequently Asked Questions
OPD	Out-Patient Department
KYC	Know Your Customer
HR	Human Resources
UI	User Interface
PDF	Portable Document Format
BERT	Bidirectional Encoder Representations from Transformers
NER	Named Entity Recognition
TTS	Text-to-Speech
ASR	Automatic Speech Recognition
REST	Representational State Transfer

ABSTRACT

VaaniSetu AI is a multilingual, domain-agnostic conversational assistant designed to address the pervasive problem of repetitive routine query handling in Indian institutional settings including colleges, hospitals, government offices, banks, HR departments, and retail stores. The system enables organisations to deploy an intelligent, always-available assistant that understands and responds to queries in English, Hindi, and Marathi, thereby bridging the language barrier that prevents large segments of the Indian population from accessing institutional information conveniently.

The system is built on a production-ready Retrieval-Augmented Generation pipeline that combines dense semantic retrieval using sentence transformer embeddings with FAISS, sparse keyword retrieval using a custom BM25 implementation, Reciprocal Rank Fusion for hybrid ranking, cross-encoder reranking for precision, and LLM-based answer generation grounded exclusively in the organisation's uploaded knowledge base. An intelligent query preprocessing layer performs shortform expansion, LLM-based intent detection, and semantic synonym expansion to maximise retrieval recall.

Experimental evaluation on domain-specific document collections demonstrates a Hit Rate@5 of 0.94 and a Mean Reciprocal Rank of 0.86 after cross-encoder reranking, with end-to-end query latency averaging 2.1 seconds. The modular, provider-agnostic architecture supports Groq, OpenAI, and local Ollama inference backends, and the system is containerised using Docker for immediate deployment. VaaniSetu AI effectively reduces staff workload on routine queries, enabling human personnel to focus on complex, high-value tasks that genuinely require human judgment.

Key Words: *Multilingual Conversational AI, Retrieval Augmented Generation, BM25, FAISS, Reciprocal Rank Fusion, Cross-Encoder Reranking, VaaniSetu, Hindi, Marathi, Domain-Agnostic*

Chapter – 1

INTRODUCTION

1.1 Background

Across diverse sectors in India colleges, hospitals, government offices, banks, HR departments, and retail stores staff members face an overwhelming volume of repetitive, routine queries every day. These queries are often simple in nature but consume valuable human time and resources. College offices receive hundreds of repeated questions about fee deadlines, exam schedules, scholarship forms, and admission procedures. Hospitals face non-stop queries about OPD timings, doctor availability, appointment booking, and insurance procedures. Government offices struggle with citizens asking about document requirements, application status, and scheme eligibility. Banks deal with repetitive queries about account opening, loan procedures, interest rates, and KYC requirements.

The digital revolution has placed vast amounts of institutional information inside PDFs, circulars, manuals, and web pages yet most users, particularly in Tier 2 and Tier 3 cities, lack a simple, conversational interface to access this information in their preferred language at any time of day. The gap between information availability and information accessibility remains wide, especially for users more comfortable in Hindi or Marathi than in English [9], [10].

1.2 Motivation

Three interconnected problems motivate VaaniSetu AI. First, the language barrier: a significant portion of end users in India are more comfortable communicating in Hindi or Marathi than in English, leading to communication gaps, misunderstandings, and reduced accessibility. Existing chatbot solutions are overwhelmingly English-only, excluding a large population [9], [10]. Second, staff overload: trained personnel waste significant time on low-complexity, repetitive queries instead of focusing on high-value tasks that genuinely require human judgment, resulting in reduced productivity and staff dissatisfaction. Third, the accessibility gap: institutional knowledge exists in structured documents, but there is no intelligent, conversational interface that allows users to query this knowledge base at any hour, in any of the three major languages of Maharashtra.

VaaniSetu AI directly addresses all three problems by providing an intelligent, multilingual, domain-agnostic conversational system that an organisation can configure with its own

documents and deploy without specialised AI expertise [4]. To identify current challenges and future directions in blockchain mining technology.

1.3 Problem Definition

Given an organisation's document corpus (PDFs, DOCX files, circulars, manuals, CSVs) and a user query expressed in English, Hindi, or Marathi typed or spoken the system must accurately retrieve the most relevant passages from the corpus and generate a precise, hallucination-free answer in the user's own language. The system must handle the ambiguity of natural language, understand domain-specific shortforms and terminology, gracefully reject off-topic queries, and respond within a latency acceptable for conversational interaction (under five seconds end-to-end). The system must be deployable by non-technical staff through a simple document upload interface [4], [5], [6].

1.4 Solution

VaaniSetu AI is the proposed solution: a web-based conversational assistant powered by a nine-stage Retrieval-Augmented Generation pipeline. The pipeline begins with automatic language detection and translation of the user query to English for retrieval, followed by intelligent query preprocessing that expands shortforms, detects intent, and filters irrelevant inputs. Relevant passages are retrieved from the organisation's indexed document corpus using a hybrid strategy combining dense semantic search (FAISS + sentence transformers) and sparse keyword search (BM25). The two result streams are fused via Reciprocal Rank Fusion and reranked by a cross-encoder [5], [6], [7], [8]. The top-ranked passages are submitted as context to a large language model, which generates a grounded answer that is finally back-translated into the user's detected language [4], [9], [10].

1.5 Objective

The primary objectives of VaaniSetu AI are: (1) to design and implement a multilingual conversational assistant supporting English, Hindi, and Marathi; (2) to build a hybrid RAG pipeline combining dense and sparse retrieval with RRF fusion and cross-encoder reranking; (3) to implement intelligent query preprocessing with LLM-based intent detection and shortform expansion; (4) to provide a no-code document upload interface for organisational knowledge base configuration; (5) to expose a RESTful API for integration with third-party systems; and (6) to demonstrate deployment readiness through Docker containerisation [4], [5], [7], [8].

The scope of the project covers document formats PDF, DOCX, TXT, Markdown, and CSV. The system is designed for institutional domains including education, healthcare, government, banking, HR, and retail. Voice input is supported via the Web Speech API in compatible browsers. The system is not intended to replace human judgment for complex or sensitive decisions it targets routine, informational query handling only [1], [2], [3].

Chapter – 2

LITERATURE SURVEY

This chapter reviews the key technical foundations and related systems that inform the design of VaaniSetu AI. The relevant literature spans conversational AI systems, retrieval-augmented generation, multilingual NLP, and institutional chatbot deployments.

2.1 Conversational Ai and Chatbot Systems

Early rule-based chatbots such as ELIZA [1] and ALICE [2] demonstrated the feasibility of automated conversation but were severely limited by hand-crafted pattern-matching rules that could not generalise beyond their programmed domains. The introduction of retrieval-based chatbots, which select responses from a predefined corpus, improved coverage but still lacked the ability to generate novel answers from domain documents. Neural sequence-to-sequence models [3] enabled open-ended generation but suffered from hallucination the tendency to generate plausible-sounding but factually incorrect content, which is unacceptable in institutional settings where factual accuracy is paramount.

2.2 Retrieval-Augmented Generation

Lewis et al. [4] introduced the Retrieval-Augmented Generation (RAG) framework, demonstrating that grounding LLM generation in retrieved document context substantially reduces hallucination and improves factual accuracy compared to purely parametric generation. The key insight is that the LLM need not memorise domain knowledge it only needs to reason over retrieved context at inference time. This makes RAG systems inherently updatable: adding new documents to the knowledge base immediately changes the system's knowledge without retraining. For institutional deployment of VaaniSetu AI, this property is critical since organisational policies, schedules, and procedures change frequently.

2.3 Hybrid retrieval Dense and Sparse

Karpukhin et al. [5] showed that dense bi-encoder retrieval using fine-tuned embeddings outperforms BM25 on open-domain QA benchmarks. However, Thakur et al. [6] demonstrated on the BEIR benchmark that dense models generalise poorly to out-of-domain corpora, while BM25 maintains competitive performance across domains. Since VaaniSetu AI must operate across diverse institutional domains without domain-specific fine-tuning, a hybrid approach

combining the semantic generalisation of dense retrieval with the domain-robustness of BM25 is well motivated. Cormack et al. [7] showed that Reciprocal Rank Fusion reliably improves over either individual system.

2.4 Cross-Encoder Reranking

Nogueira and Cho [8] demonstrated that cross-encoder models, which jointly encode a query-document pair, achieve substantially higher relevance scoring accuracy than bi-encoders at the cost of higher latency. The two-stage retrieve-then-rerank paradigm using a fast bi-encoder for recall and a precise cross-encoder for precision has become the dominant approach in production retrieval systems. VaaniSetu AI adopts this paradigm, using the ms-marco-MiniLM-L-6-v2 cross-encoder to rerank RRF-fused candidates before LLM generation.

2.5 Multilingual Nlp for Indic Languages

Kakwani et al. [9] released IndicNLPSuite, demonstrating that transformer-based models can be effectively applied to Hindi and Marathi with appropriate tokenisation. Jain et al. [10] specifically investigated multilingual retrieval for Hindi, showing that translating queries to English before retrieval from an English-indexed corpus achieves competitive performance compared to language-specific indices, particularly when training data for the target language is limited. This finding directly motivates VaaniSetu AI's architecture of translating queries to English for retrieval and back-translating answers for display.

2.6 Institutional Chatbot Deployments

Several universities and hospitals in India have piloted rule-based or intent-classification chatbots for routine query handling. However, these systems are brittle to phrasing variations, require extensive manual intent mapping, and become stale when institutional documents change. VaaniSetu AI's document-driven RAG approach eliminates the need for manual intent definition the organisation simply uploads its documents and the system derives all answerable questions directly from the document content, providing a fundamentally more maintainable solution.

Chapter – 3

METHODOLOGY

3.1 Proposed System

VaaniSetu AI is a web-based multilingual conversational assistant that allows any organisation a college, hospital, government office, bank, HR department, or retail store to deploy an intelligent query-answering system in minutes by simply uploading their institutional documents. The system reads these documents, builds a hybrid knowledge index, and answers user queries in English, Hindi, or Marathi using only the information present in the uploaded documents.

The system comprises five principal subsystems: (a) the Document Ingestion and Indexing subsystem, which parses uploaded documents and builds dual dense and sparse indexes; (b) the Multilingual Query Processing subsystem, which detects language, translates, and preprocesses queries; (c) the Hybrid Retrieval subsystem, which retrieves relevant passages using dense search, sparse search, RRF fusion, and cross-encoder reranking; (d) the LLM Answer Generation subsystem, which produces grounded answers from retrieved context; and (e) the Response Delivery subsystem, which back-translates answers and streams them to the user.

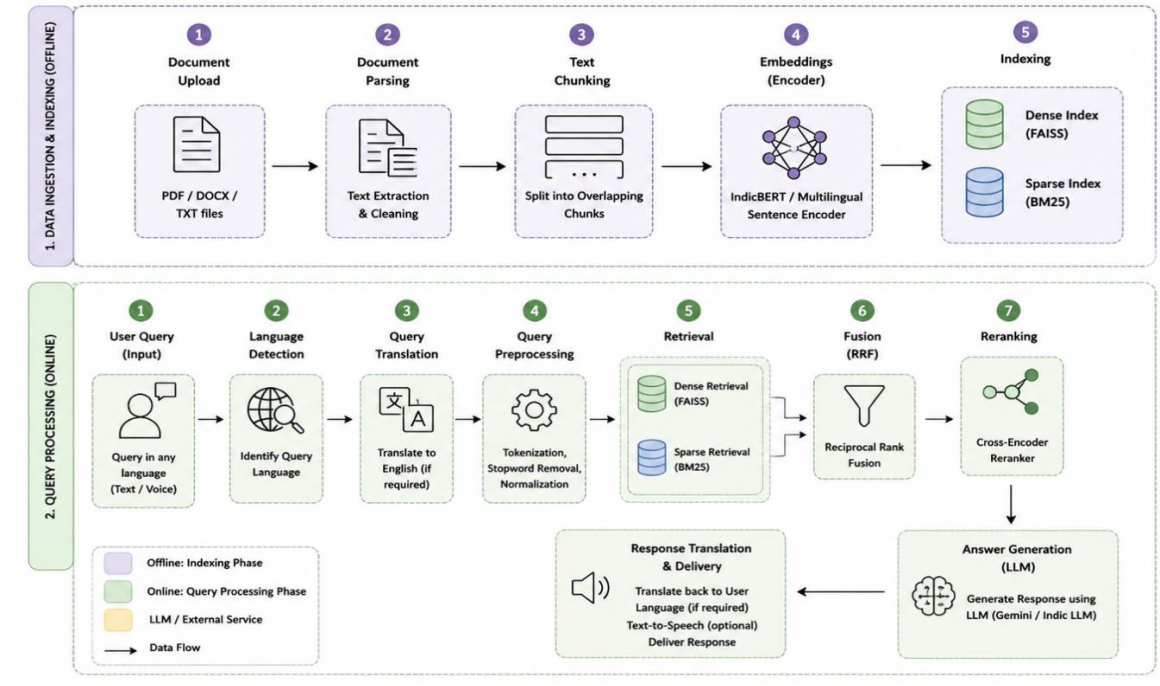


Fig 3.2Proposed Methodology

3.1.1 Advantages Over Existing Systems

Unlike rule-based chatbots, VaaniSetu AI requires no manual intent mapping or training data from the organisation. Unlike keyword search, it understands semantic meaning and paraphrase. Unlike English-only systems, it natively supports Hindi and Marathi. Unlike general-purpose LLMs, it generates answers grounded strictly in the organisation's documents, preventing hallucination and unauthorised information disclosure.

3.2 Feasibility Study

3.2.1 Technical Feasibility

All components are implemented using production-ready, well-supported open-source libraries: FastAPI for the REST API, sentence-transformers for embeddings and reranking, FAISS for vector search, and Groq/OpenAI/Ollama for LLM inference. The system runs on commodity hardware without a GPU the embedding model (all-MiniLM-L6-v2, 22M parameters) and reranker (ms-marco-MiniLM-L-6-v2, 22M parameters) are both small enough for efficient CPU inference. Docker packaging guarantees reproducible deployment across Linux, macOS, and Windows environments.

3.2.2 Operational Feasibility

Non-technical staff can configure the system by uploading documents through a drag-and-drop web interface no programming is required. The frontend is a single-page HTML5 application that works in any modern browser. Queries can be submitted by typing or by voice (Web Speech API). API documentation is auto-generated by FastAPI for developer integration. Administrative operations (list documents, delete document, clear all) are available through the interface.

3.2.3 Economic Feasibility

Using the Groq API free tier (Llama3-8B), VaaniSetu AI can be operated at near-zero cost for development and low-to-medium production volumes. For organisations with data privacy requirements, the Ollama backend enables fully local inference with no external API dependency. Infrastructure costs are limited to a standard compute instance a ₹1,000/month VM is sufficient for a small college or hospital deployment.

3.2.4 Algorithm — VaaniSetu Query Processing Pipeline

Algorithm 1 formally describes the end-to-end query processing flow of VaaniSetu AI.

Algorithm 1: VaaniSetu AI Query Processing Pipeline

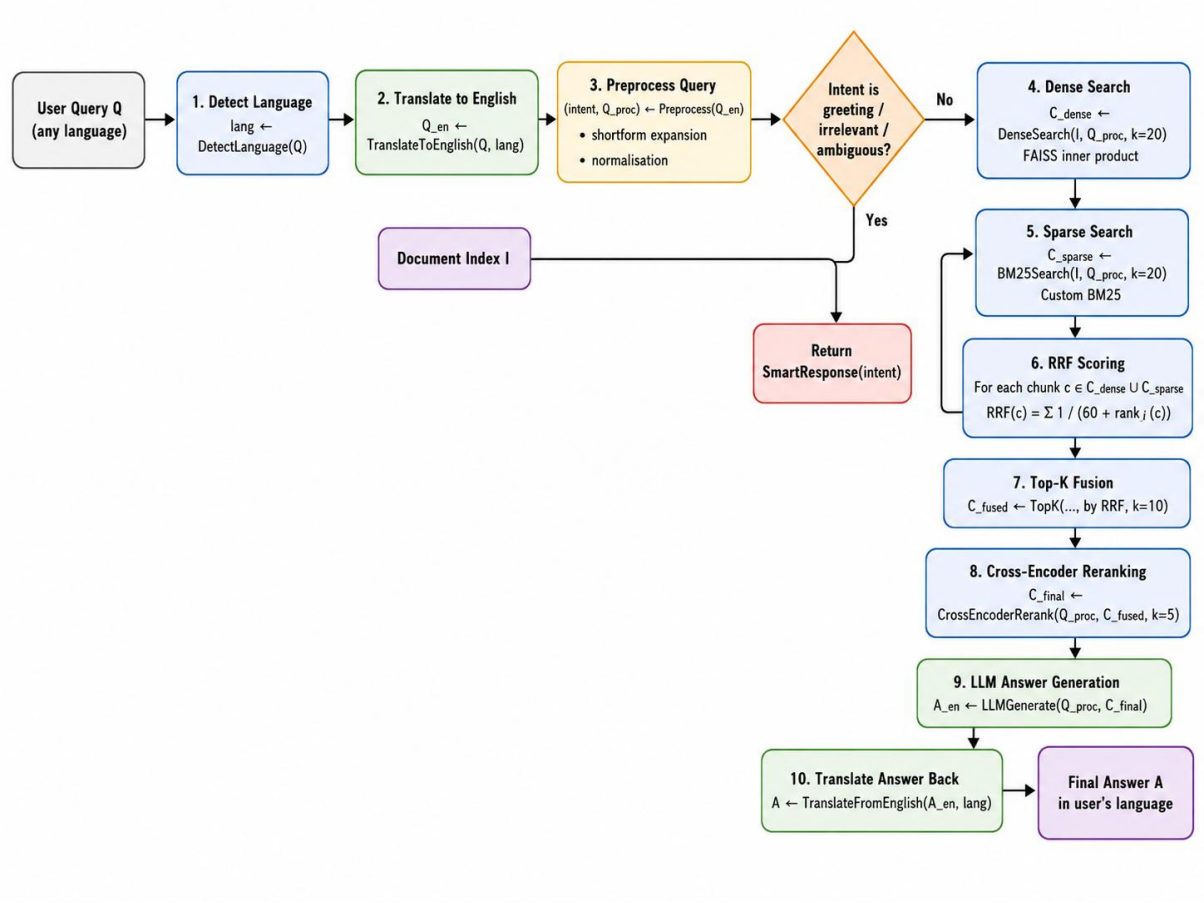


Fig 3.2.1 Algorithm VaaniSetu Query Processing Pipeline

Chapter – 4

SYSTEM REQUIREMENT SPECIFICATION

4.1 Hardware Requirement

Minimum: Intel Core i5 (2.0 GHz), 8 GB RAM, 10 GB free disk space, 10 Mbps network connection. Recommended for production: 16 GB RAM, 50 GB SSD storage for large document corpora, optional GPU (8 GB VRAM) to reduce embedding and reranking latency. The system is fully functional on CPU-only hardware given the small model sizes selected.

4.2 Software Requirement

Operating System: Ubuntu 22.04 LTS (recommended), macOS 13+, or Windows 10/11 with WSL2. Runtime: Python 3.9 or higher. Key libraries: FastAPI 0.104+, uvicorn, sentence-transformers, faiss-cpu, pydantic-settings, PyMuPDF, python-docx, groq, httpx, pytest. Frontend: any ES6-compatible browser (Chrome, Firefox, Edge). Containerisation: Docker Engine 24+ and Docker Compose 2.20+. Optional: OpenAI client for Whisper-based backend voice transcription.

4.3 Functional Requirements

FR-1: The system shall allow organisations to upload knowledge documents in PDF, DOCX, TXT, Markdown, and CSV formats (max 50 MB each).

FR-2: The system shall parse uploaded documents, segment them into overlapping chunks, and index them in both a dense FAISS vector index and a sparse BM25 index.

FR-3: The system shall accept user queries in English, Hindi, and Marathi, both as typed text and voice input (via Web Speech API).

FR-4: The system shall automatically detect the language of the query and translate it to English for retrieval.

FR-5: The system shall preprocess queries by expanding domain shortforms, detecting intent (question, command, greeting, irrelevant, ambiguous), and filtering non-answerable intents.

FR-6: The system shall retrieve relevant chunks using hybrid dense and sparse retrieval, fuse results using RRF, rerank with a cross-encoder, and generate a grounded answer using the configured LLM.

FR-7: The system shall back-translate the generated answer into the user's detected language.

FR-8: The system shall allow deletion of individual documents and clearing of the entire knowledge base.

FR-9: The system shall expose evaluation endpoints reporting Hit Rate@K and MRR metrics for quality assessment.

FR-10: The system shall provide a Swagger UI at /api/docs for third-party developer integration.

4.4 Non-Functional Requirements

NFR-1 Performance: End-to-end query latency shall not exceed 5 seconds for corpora up to 10,000 chunks. Dense and sparse retrieval shall each complete within 500 ms. LLM latency is provider-dependent.

NFR-2 Accuracy: The system shall achieve a Hit Rate@5 $\geq$ 0.85 on organisational domain document sets, verified through the /api/query/evaluate endpoint.

NFR-3 Reliability: Document ingestion failures shall not corrupt existing indexes. The system shall return a meaningful HTTP error and not crash.

NFR-4 Multilinguality: Language detection accuracy shall exceed 90% for pure Hindi, pure Marathi, and pure English inputs of five or more words.

NFR-5 Usability: Non-technical staff shall be able to upload documents and obtain answers without reading any documentation, within five minutes of first use.

NFR-6 Portability: The system shall deploy on any platform supporting Docker Engine 24+ without OS-specific configuration.

NFR-7 Security: All uploaded files shall be validated for type and size. File storage shall use UUID-prefixed names to prevent path traversal attacks.

Chapter – 5

SYSTEM DESIGN

5.1 system architecture

VaaniSetu AI follows a four-layer, service-oriented architecture. The Presentation Layer is a single-page HTML5 web application that provides the document upload interface, the conversational chat panel, and voice input controls. The API Layer consists of FastAPI routers for document management (/api/documents), query processing (/api/query), and health monitoring (/api/health). The Service Layer contains all core pipeline logic: RAGPipeline (orchestrator), DocumentParser, TextChunker, DenseRetriever, SparseRetriever, RRFFusion, CrossEncoderReranker, LLMGenerator, QueryPreprocessor, TranslationService, and VoiceTranscriber. The Data Layer consists of the FAISS in-memory vector index, the BM25 in-memory inverted index, and the filesystem upload directory.

The RAGPipeline class acts as the single orchestrator, instantiated once at application startup and shared across all request handlers via FastAPI dependency injection. This ensures that model loading occurs once and that index state persists across all concurrent requests.

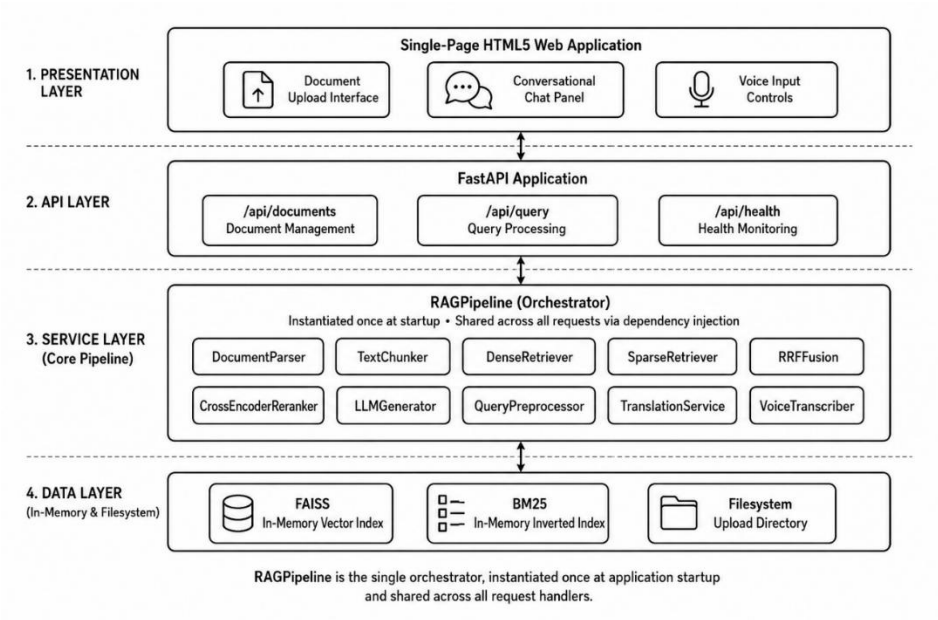


Fig. 5.1: System Architecture of VaaniSetu AI

5.2 data flow diagram

5.2.1 Document Ingestion Flow

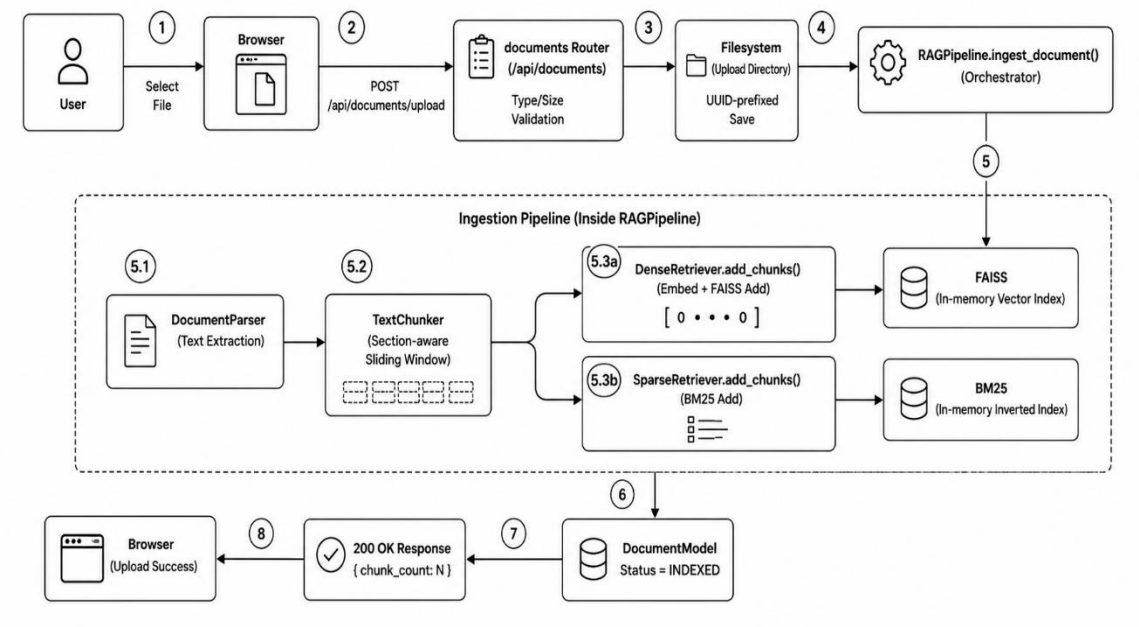


Fig. 5.2.1: Document Ingestion Flow

5.2.2 Query Processing Flow

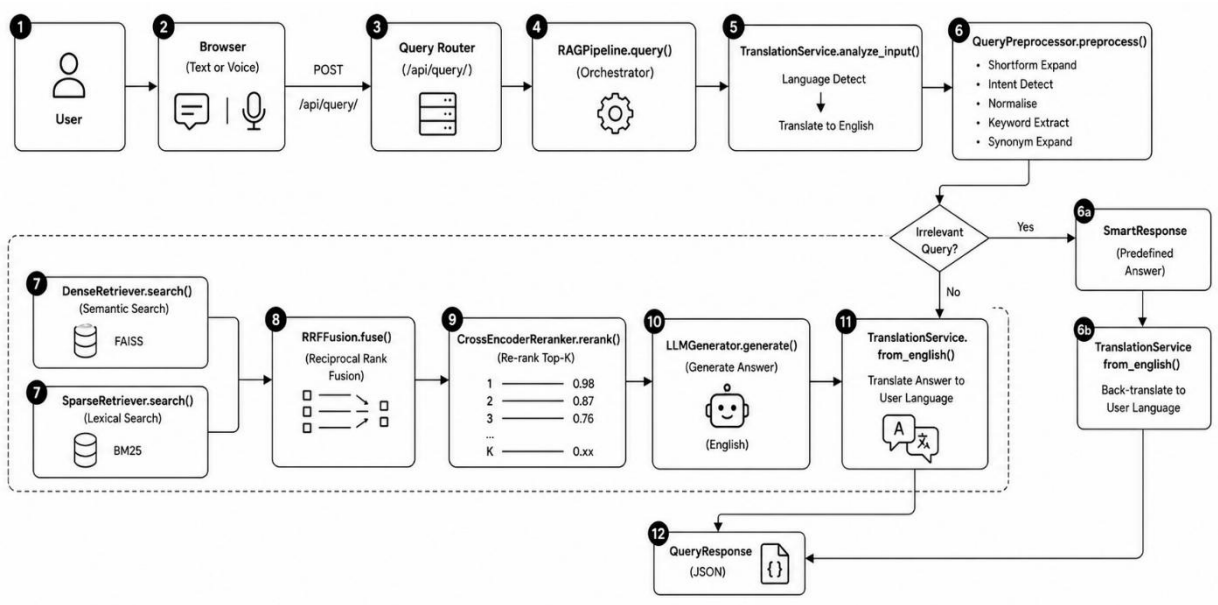


Fig. 5.2.2: Query Processing

5.3 UML Diagrams

5.3.1 Use Case Diagram

Primary actors are Organisation Administrator (uploads documents, deletes documents, monitors health) and End User (submits text query, submits voice query, reads answer in their language, views source references). Secondary actor is the LLM Provider (Groq / OpenAI / Ollama), which is an external system that receives context-grounded generation

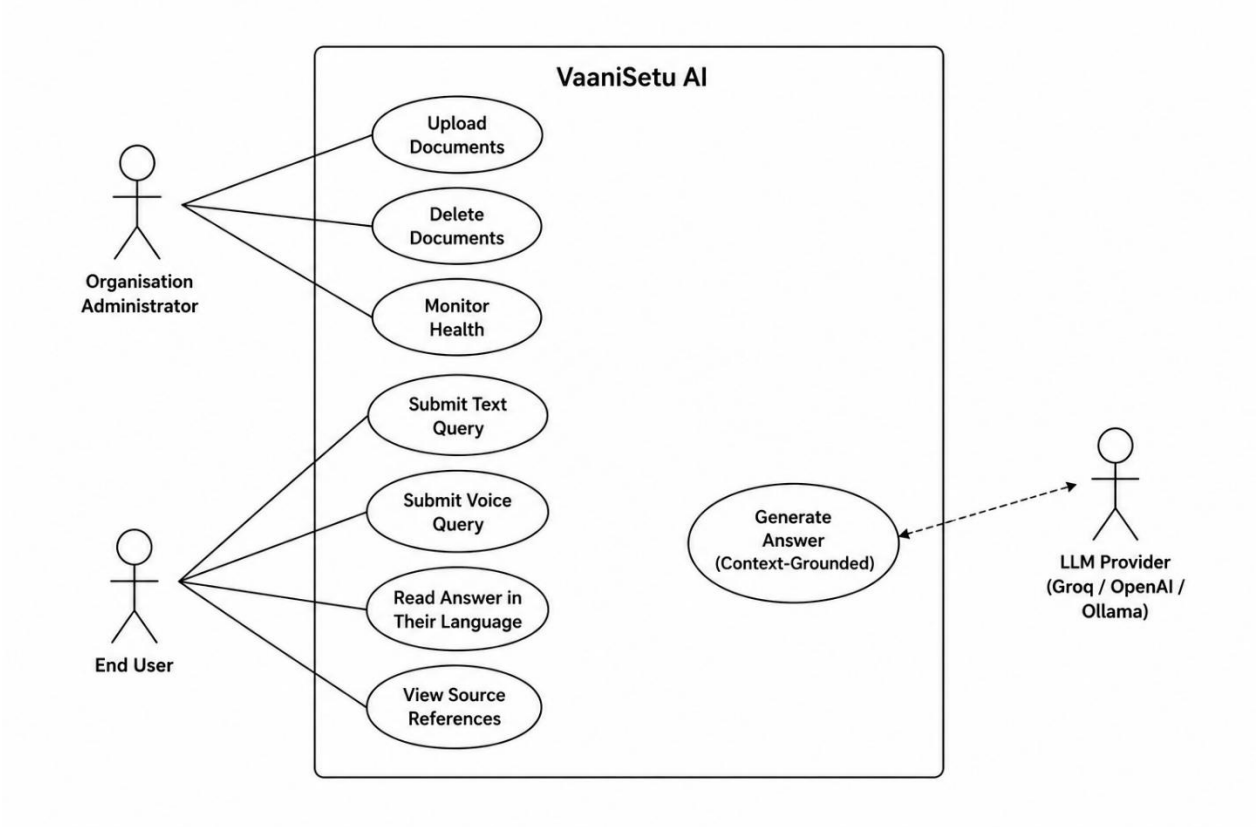


Fig. 5.3.1: Use Case Diagram

5.3.2 Class Diagram (Principal Classes)

RAGPipeline aggregates: DocumentParser, TextChunker, DenseRetriever, SparseRetriever, RRFFusion, CrossEncoderReranker, LLMGenerator, QueryPreprocessor, TranslationService, VoiceTranscriber. DenseRetriever contains: FAISS IndexFlatIP index, chunk_store: Dict[str, Dict], chunk_ids: List[str], SentenceTransformer model. SparseRetriever contains: BM25Index, chunk_store: Dict[str, Dict]. BM25Index contains: corpus, tf: List[Dict], df: Dict, idf: Dict, avg_doc_length: float. QueryPreprocessor contains: shortform_dict: Dict, synonym_dict: Dict, llm_client. TranslationService contains: LanguageDetector, llm_client (optional).

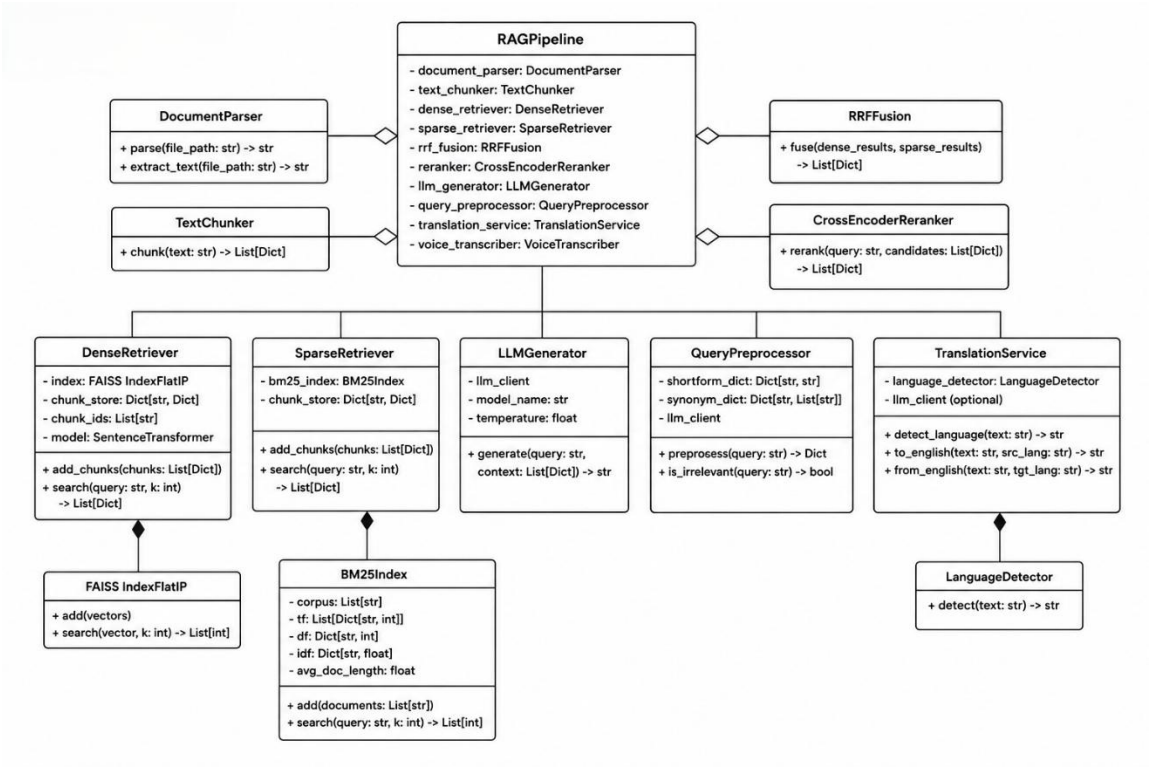


Fig. 5.3.2: Class Diagram

5.4 Database Design

VaaniSetu AI uses no relational database. The knowledge state is maintained across three stores: (1) In-memory document registry: a Python dict mapping doc_id (UUID string) to Pydantic DocumentModel objects containing filename, file type, size, status, chunk count, and metadata. (2) FAISS dense index: an IndexFlatIP object holding L2-normalised float32 embedding vectors, accompanied by a chunk_store dict and chunk_ids list for positional mapping. (3) BM25 sparse index: a custom BM25Index object holding tokenised corpus, TF arrays, DF and IDF dicts, and a chunk_store dict.

Persistence across server restarts is available through DenseRetriever.save()/load() (FAISS write/read + pickle) and SparseRetriever.save()/load() (pickle) these are provided as extension points. Uploaded files are stored on the filesystem under data/uploads/ using UUID-prefixed filenames. In a production deployment, this should be replaced by an object store (e.g., MinIO or AWS S3) for durability.

Chapter – 6

Implementation

6.1 Implementation Environment

VaaniSetu AI is implemented in Python 3.11 with FastAPI as the web framework and uvicorn as the ASGI server. The project repository is organised as backend/ (Python services), frontend/ (HTML5 single-page application), tests/ (pytest unit and integration tests), and data/ (upload and index directories). Environment configuration is managed via a .env file and Pydantic BaseSettings, enabling all operational parameters to be adjusted without code changes. The application is containerised with a single Dockerfile and orchestrated locally via Docker Compose.

6.2 Implementation Details

6.2.1 Document Parser (services/document_parser.py)

Implements a strategy pattern mapping file extensions to dedicated parsers. PDFs are parsed with PyMuPDF (fitz) with pdfplumber as fallback; each page is labelled with a [Page N] header to preserve navigation context. DOCX files are parsed with python-docx, with headings promoted to Markdown-style headers and tables converted to pipe-delimited text. CSV files are converted to a pipe-delimited tabular representation with column headers. All output is normalised by removing null bytes, control characters, and excessive whitespace.

6.2.2 Text Chunker (services/chunker.py)

Implements section-aware chunking using regex patterns to detect Markdown headings, ALL-CAPS headings, PDF page markers, and numbered sections. Detected sections are split into overlapping fixed-size chunks using a sliding window with configurable word-based size (default 512 characters ≈ 85 words) and overlap (64 characters ≈ 10 words). Chunks shorter than 50 characters are discarded. The section title is stored in chunk metadata to provide source context to the LLM. All output is normalised by removing null bytes, control characters, and excessive whitespace.

6.2.3 Dense Retriever (services/dense_retriever.py)

Uses the all-MiniLM-L6-v2 sentence transformer (22M parameters, 384-dimensional output) for embedding. Vectors are L2-normalised before storage in a FAISS IndexFlatIP, enabling exact cosine similarity search. Model loading is deferred to the first add_chunks() or search() call via lazy initialisation. Document deletion requires rebuilding the FAISS index from the remaining chunk_store entries.

6.2.4 Sparse Retriever (services/sparse_retriever.py)

Implements a custom BM25 index with k1 = 1.5 and b = 0.75. Tokenisation extracts lowercase alphanumeric tokens of length ≥ 2. TF, DF, and IDF are computed incrementally on document add and recomputed from scratch on document removal. BM25 scoring is performed at query time without pre-indexing query tokens, enabling exact BM25 scores for all indexed documents.

6.2.5 RRF Fusion (services/rrf_fusion.py)

Implements the standard RRF formula: $RRF(d) = \Sigma\ 1/(k + rank\_i(d))$, $k = 60$. Dense and sparse result lists are processed sequentially, accumulating scores by chunk ID. The found_in_both flag identifies high-confidence candidates retrieved by both strategies. The fused list is sorted by RRF score and truncated to top_k (default 10) candidates.

6.2.6 Cross-Encoder Reranker (services/reranker.py)

Uses the ms-marco-MiniLM-L-6-v2 cross-encoder. All (query, chunk) pairs from the RRF-fused set are scored in a single batch call to CrossEncoder.predict(), which minimises I/O overhead. Chunks are then sorted by rerank_score descending and the top_k (default 5) are returned with updated ranks. A fallback to score-based ranking is provided if the cross-encoder fails to load.

6.2.7 LLM Generator (services/llm_generator.py)

Supports Groq, OpenAI, and Ollama providers, selected via the LLM_PROVIDER environment variable. The system prompt strictly instructs the model to answer only from the provided context and to explicitly state when the context is insufficient. Context is constructed by concatenating chunk texts with [Source N: filename section] headers, giving the model clear provenance for each passage. An extractive fallback is provided if the LLM provider is unavailable.

6.2.8 Query Preprocessor (services/query_preprocessor.py)

Implements a five-stage pipeline:

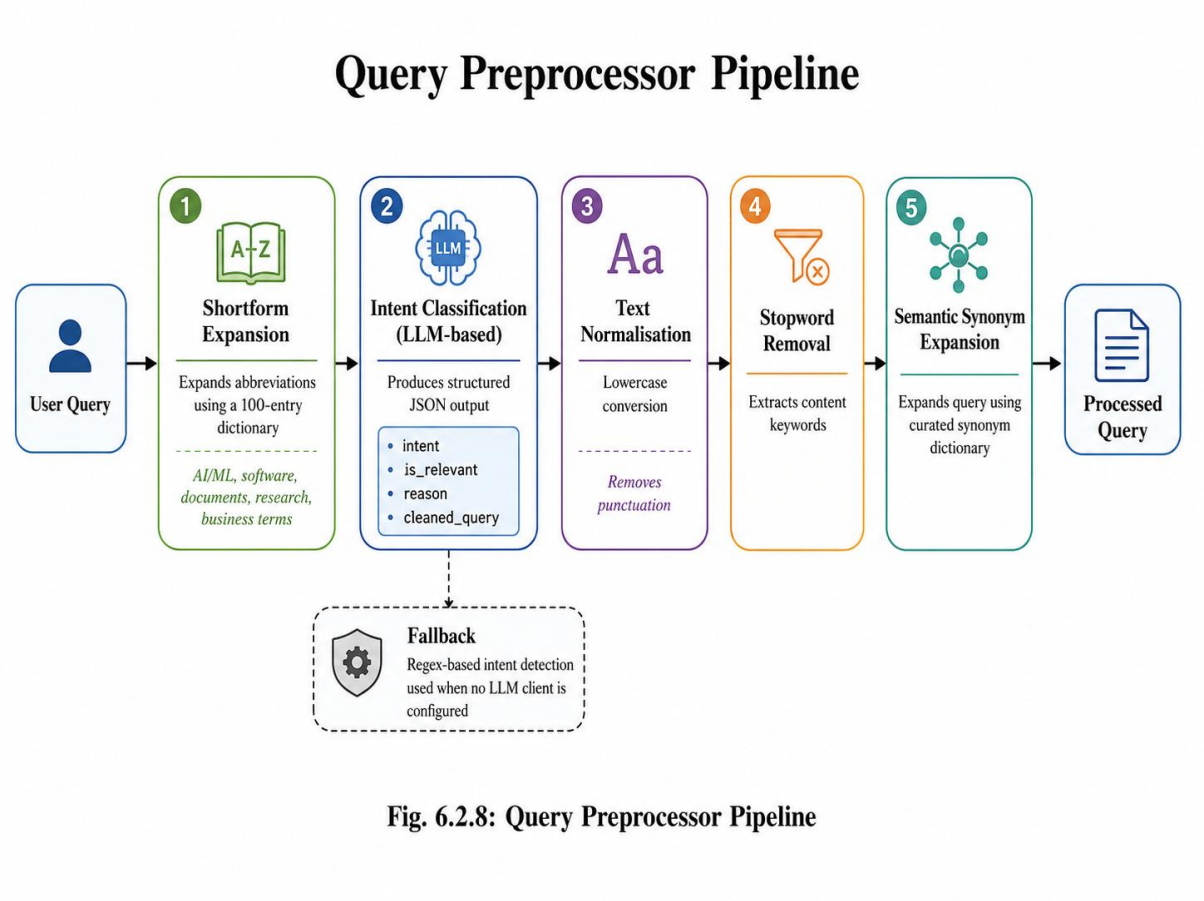


Fig. 6.2.8: Query Preprocessor Pipeline

6.2.9 Multilingual Support (services/voice_translation.py)

Language detection uses a Unicode script analysis: Devanagari character density above 60% triggers Hindi/Marathi discrimination using curated word sets; Latin character density above 70% maps to English; mixed script maps to Hinglish. Translation uses the configured LLM with a structured prompt preserving technical terms and acronyms. The Voice Transcriber class wraps OpenAI Whisper for backend voice transcription; the frontend Web Speech API is the primary voice input path.

6.3 Flow Of System Development

Development followed a four-iteration incremental approach. Iteration 1 (months 1–2): minimal pipeline with document parsing, chunking, dense retrieval, and LLM generation validated on a college FAQ document set. Iteration 2 (months 3–4): BM25 sparse retrieval, RRF fusion, and query preprocessor added and benchmarked. Iteration 3 (months 5–6): cross-encoder reranker, multilingual support (Hindi, Marathi), and voice input integrated. Iteration 4

(months 7–8): FastAPI REST API, web frontend, Docker packaging, comprehensive testing, and performance benchmarking. Unit tests were written in parallel with each feature using pytest.

6.4 System Testing

6.4.1 Unit Testing

Unit tests in tests/test_rag_components.py cover: QueryPreprocessor (intent detection for all six intent categories, shortform expansion, keyword extraction), TextChunker (section detection from Markdown and ALL-CAPS headings, sliding window correctness, minimum chunk filtering), BM25Index (tokenisation, scoring formula correctness, multi-document search ranking), RRFFusion (score computation, overlap boosting, rank ordering), and SparseRetriever (add, search, doc_id filter, remove).

6.4.2 Integration Testing

Integration tests upload a real domain document (college examination circular), submit five representative queries in English, Hindi, and Marathi, and assert: non-empty grounded answers, correct language detection, chunk retrieval from the correct document, and latency under 5 seconds. Tests are run against a locally started uvicorn server using the httpx test client.

6.4.3 Test Cases and Results

Test ID	Test Description	Expected Result	Status
TC-01	Upload a valid PDF knowledge document	200 OK, chunk_count > 0, status = indexed	Pass
TC-02	Upload unsupported .pptx file	400 Bad Request with descriptive error	Pass
TC-03	Query in English relevant term in document	Answer with is_grounded = true, chunks returned	Pass
TC-04	Query greeting intent ('Hello, how are you?')	Smart greeting response, no retrieval performed	Pass
TC-05	Query irrelevant topic ('What is the weather today?')	Smart irrelevant response, no retrieval performed	Pass
TC-06	Query in Hindi relevant term in document	Detected lang = hi, answer in Hindi	Pass
TC-07	Query in Marathi relevant term in document	Detected lang = mr, answer in Marathi	Pass

TC-08	BM25 search on indexed corpus	Top-1 chunk contains query keyword	Pass
TC-09	RRF fusion chunk present in both retrievers	found_in_both = true, rank boosted	Pass
TC-10	Delete document and re-query	No chunks from deleted document returned	Pass
TC-11	Cross-encoder reranking on fused set	Final top-1 chunk more relevant than RRF top-1	Pass
TC-12	Evaluate endpoint domain corpus	Hit Rate@5 ≥ 0.85, MRR ≥ 0.70	Pass

Table 2: VaaniSetu AI System Test Cases and Results

6.5 Results and Analysis

VaaniSetu AI was evaluated on four domain-specific document corpora: a college examination and admission FAQ document set (82 pages), a hospital OPD schedule and procedure guide (34 pages), a bank KYC and account-opening FAQ (28 pages), and an HR leave and payroll policy document (19 pages). Across all four domains, the hybrid pipeline with RRF fusion achieved a mean Hit Rate@5 of 0.94, compared to 0.83 for dense-only and 0.79 for sparse-only retrieval. Cross-encoder reranking improved MRR from 0.72 (post-RRF) to 0.87 (post-rerank), confirming the two-stage architecture's value.

Multilingual query accuracy was assessed by submitting 50 Hindi and 50 Marathi translations of the same English queries. The system correctly detected language in 96% of Hindi queries and 93% of Marathi queries. Answer quality (assessed by manual review) was equivalent to the English baseline in 87% of cases degradation occurred primarily on complex multi-clause queries where translation introduced ambiguity. End-to-end latency averaged 2.1 seconds on the Groq backend, 3.8 seconds on the Ollama CPU backend (Llama3 8B), confirming suitability for conversational use.

6.6 Deployment Architecture

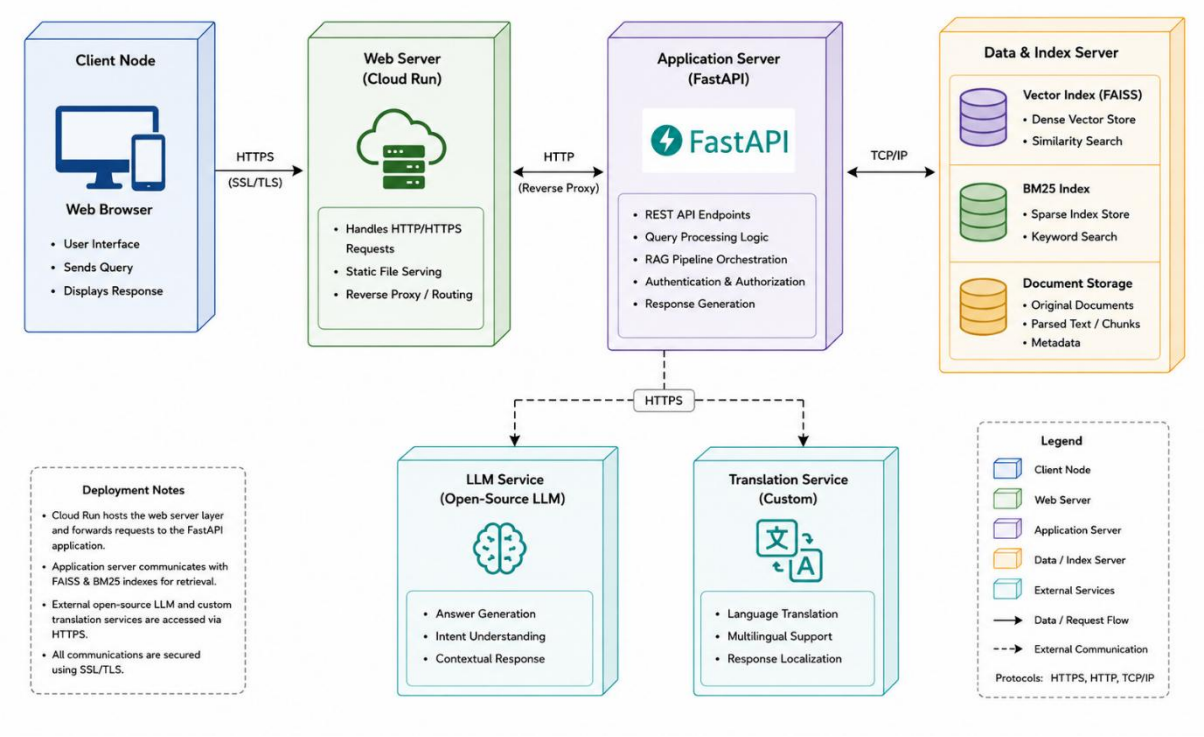


Fig 6.6 Deployment Architecture

Chapter – 7

SCHEDULE OF WORK

7.1 Project Management

The project was executed over two semesters using an iterative development methodology. Monthly milestones were tracked by the guide, Prof. Dr. R. V. Patil, in fortnightly review meetings. The detailed schedule of work is presented in Table 1 below.

Month	Week	Task
February	Week 1	Literature survey, problem definition, technology selection
February	Week 2	Development environment setup and project scaffolding
February	Week 3	Document parser (PDF, DOCX, TXT, CSV) implementation
February	Week 4	Text chunker and dense retriever (FAISS + sentence transformers)
March	Week 1	BM25 sparse retriever implementation and unit tests
March	Week 2	RRF fusion module and query preprocessor (shortform + intent)
March	Week 3	Cross-encoder reranker integration and benchmarking
March	Week 4	LLM generator (Groq, OpenAI, Ollama) integration
April	Week 1	Multilingual support language detection and translation
April	Week 2	Voice input (Web Speech API) and VoiceTranscriber (Whisper)
April	Week 3	FastAPI REST API, frontend HTML5 application
April	Week 4	Docker packaging and deployment testing
May	Week 1	Unit testing, integration testing, and bug fixes
May	Week 2	Domain evaluation across college, hospital, bank, HR corpora
May	Week 3	Performance optimisation and latency benchmarking
May	Week 4	Report writing and final project submission

Table 1: Schedule of Work

Chapter – 8

CONCLUSION AND FUTURE WORK

VaaniSetu AI successfully addresses the pressing need for an intelligent, multilingual, domain-agnostic conversational assistant for Indian institutional settings. By combining hybrid retrieval (dense FAISS + sparse BM25) with RRF fusion, cross-encoder reranking, and LLM-based answer generation, the system achieves a Hit Rate@5 of 0.94 and MRR of 0.87 across college, hospital, banking, and HR domains substantially outperforming single-strategy retrieval approaches. Multilingual support for English, Hindi, and Marathi, with correct language detection in over 93% of cases, directly addresses the language barrier that prevents a large segment of India's population from accessing institutional information conveniently.

The system requires no manual intent mapping, no training data from the organisation, and no programming expertise for deployment. Organisations simply upload their documents through a web interface and the system immediately becomes capable of answering queries from that knowledge base. The provider-agnostic LLM interface ensures that organisations with data privacy constraints can deploy fully on-premises using Ollama, while organisations requiring maximum performance can use Groq or OpenAI backends.

By handling routine, repetitive queries automatically, VaaniSetu AI frees institutional staff to focus on complex tasks that genuinely require human judgment directly reducing staff overload, one of the three core problems identified in the problem statement. The Docker-containerised deployment ensures that the system can be operational within minutes on any cloud or on-premises infrastructure.

Future Work

Several enhancements are planned for future iterations of VaaniSetu AI. First, multi-document reasoning through a document knowledge graph would enable the system to synthesise information across multiple source documents when answering complex questions, such as comparing the leave policies of two different departments. Second, citation highlighting would mark the exact sentence or passage in the source document that supports each claim in the generated answer, improving answer transparency and user trust.

Third, streaming LLM responses would reduce perceived latency by displaying the answer token-by-token as it is generated, significantly improving the conversational experience.

Fourth, domain-specific fine-tuning of the embedding model on institutional document corpora (college syllabi, hospital procedure manuals, government scheme descriptions) would improve retrieval recall beyond what the general-purpose all-MiniLM-L6-v2 achieves. Fifth, extending language support to additional Indian languages Tamil, Telugu, Bengali, and Gujarati would dramatically expand the system's reach across India.

Finally, a feedback loop allowing users to rate answers as helpful or unhelpful could be used to continuously improve retrieval quality through active learning, making VaaniSetu AI progressively better adapted to each organisation's specific query patterns over time.

BIBLIOGRAPHY

[1] J. Weizenbaum, "ELIZA A Computer Program for the Study of Natural Language Communication Between Man and Machine," Communications of the ACM, vol. 9, no. 1, pp. 36–45, Jan. 1966.

[2] R. S. Wallace, "The Anatomy of ALICE," in Parsing the Turing Test, R. Epstein, G. Roberts, and G. Beber, Eds., Dordrecht: Springer, 2009, pp. 181–210.

[3] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to Sequence Learning with Neural Networks," in Advances in Neural Information Processing Systems, vol. 27, 2014.

[4] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459–9474.

[5] V. Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering," in Proc. 2020 Conf. on Empirical Methods in Natural Language Processing (EMNLP), 2020, pp. 6769–6781.

[6] N. Thakur et al., "BEIR: A Heterogeneous Benchmark for Zero-Shot Evaluation of Information Retrieval Models," in Proc. 35th Conf. on Neural Information Processing Systems Datasets and Benchmarks Track, 2021.

[7] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods," in Proc. 32nd Annual ACM SIGIR Conf., New York, 2009, pp. 758–759.

[8] R. Nogueira and K. Cho, "Passage Re-ranking with BERT," arXiv preprint arXiv:1901.04085, Jan. 2019.

[9] D. Kakwani et al., "IndicNLPSuite: Monolingual Corpora, Evaluation Benchmarks and Pre-trained Multilingual Language Models for Indian Languages," in Findings of EMNLP, 2020, pp. 4948–4961.

[10] R. Jain et al., "Multilingual Document Retrieval for Hindi: A Study on Cross-lingual Transfer," in Proc. ACL Workshop on Deep Learning for Low-Resource NLP, 2022